

input, and corresponding elements will appear in the same order. This kind of algebraic reasoning can potentially save us a lot of work, since we don't have to write separate tests for these properties.

11.2 Monads: generalizing the flatMap and unit functions

Functor is just one of many abstractions that we can factor out of our libraries. But Functor isn't too compelling, as there aren't many useful operations that can be defined purely in terms of `map`. Next, we'll look at a more interesting interface, `Monad`. Using this interface, we can implement a number of useful operations, once and for all, factoring out what would otherwise be duplicated code. And it comes with laws with which we can reason that our libraries work the way we expect.

Recall that for several of the data types in this book so far, we implemented `map2` to “lift” a function taking two arguments. For `Gen`, `Parser`, and `Option`, the `map2` function could be implemented as follows.

Listing 11.1 Implementing `map2` for `Gen`, `Parser`, and `Option`

```
def map2[A,B,C] (
  fa: Gen[A], fb: Gen[B])(f: (A,B) => C): Gen[C] =
  fa flatMap (a => fb map (b => f(a,b)))
```

Makes a generator of a random `C` that runs random generators `fa` and `fb`, combining their results with the function `f`.

```
def map2[A,B,C] (
  fa: Parser[A], fb: Parser[B])(f: (A,B) => C): Parser[C] =
  fa flatMap (a => fb map (b => f(a,b)))
```

Makes a parser that produces `C` by combining the results of parsers `fa` and `fb` with the function `f`.

```
def map2[A,B,C] (
  fa: Option[A], fb: Option[B])(f: (A,B) => C): Option[C] =
  fa flatMap (a => fb map (b => f(a,b)))
```

Combines two `Options` with the function `f`, if both have a value; otherwise returns `None`.

These functions have more in common than just the name. In spite of operating on data types that seemingly have nothing to do with one another, the implementations are identical! The only thing that differs is the particular data type being operated on. This confirms what we've suspected all along—that these are particular instances of some more general pattern. We should be able to exploit that fact to avoid repeating ourselves. For example, we should be able to write `map2` once and for all in such a way that it can be reused for all of these data types.

We've made the code duplication particularly obvious here by choosing uniform names for our functions, taking the arguments in the same order, and so on. It may be more difficult to spot in your everyday work. But the more libraries you write, the better you'll get at identifying patterns that you can factor out into common abstractions.